UNIT V

Transmission Control Protocol (TCP)

TCP is a connection oriented protocol and offers end-to-end packet delivery. It acts as back bone for connection.It exhibits the following key features:

- Transmission Control Protocol (TCP) corresponds to the Transport Layer of OSI Model.
- TCP is a reliable and connection oriented protocol.
- TCP offers:
    - Stream Data Transfer.
    - Reliability.
    - Efficient Flow Control
    - Full-duplex operation.
    - Multiplexing.
- TCP offers connection oriented end-to-end packet delivery.
- TCP ensures reliability by sequencing bytes with a forwarding acknowledgement number that indicates to the destination the next byte the source expect to receive.
- It retransmits the bytes not acknowledged with in specified time period.

TCP Services

TCP offers following services to the processes at the application layer:

- Stream Delivery Service
- Sending and Receiving Buffers
- Bytes and Segments
- Full Duplex Service
- Connection Oriented Service
- Reliable Service

Stream Deliver Service

TCP protocol is stream oriented because it allows the sending process to send data as stream of bytes and the receiving process to obtain data as stream of bytes.

Sending and Receiving Buffers

It may not be possible for sending and receiving process to produce and obtain data at same speed, therefore, TCP needs buffers for storage at sending and receiving ends.

Bytes and Segments

The Transmission Control Protocol (TCP), at transport layer groups the bytes into a

packet. This packet is called segment. Before transmission of these packets, these segments are encapsulated into an IP datagram.

Full Duplex Service

Transmitting the data in duplex mode means flow of data in both the directions at the same time.

Connection Oriented Service

TCP offers connection oriented service in the following manner:

1. TCP of process-1 informs TCP of process – 2 and gets its approval.
2. TCP of process – 1 and TCP of process – 2 and exchange data in both the two directions.
3. After completing the data exchange, when buffers on both sides are empty, the two TCP's destroy their buffers.

Reliable Service

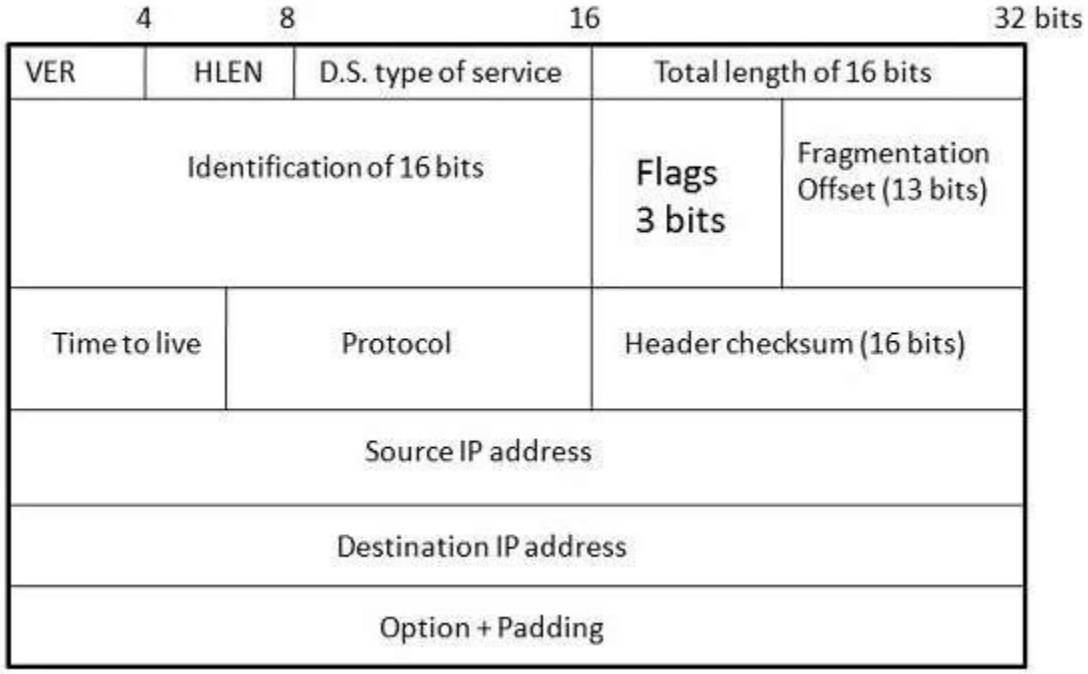
For sake of reliability, TCP uses acknowledgement mechanism.

Internet Protocol (IP)

Internet Protocol is **connectionless** and **unreliable** protocol. It ensures no guarantee of successfully transmission of data.

In order to make it reliable, it must be paired with reliable protocol such as TCP at the transport layer.

Internet protocol transmits the data in form of a datagram as shown in the following diagram:



Points to remember:

- The length of datagram is variable.
- The Datagram is divided into two parts: **header** and **data.**
- The length of header is 20 to 60 bytes.
- The header contains information for routing and delivery of the packet.

User Datagram Protocol (UDP)

Like IP, UDP is connectionless and unreliable protocol. It doesn't require making a connection with the host to exchange data. Since UDP is unreliable protocol, there is no mechanism for ensuring that data sent is received.

UDP transmits the data in form of a datagram. The UDP datagram consists of five parts as shown in the following diagram:

Source Port	Destination Port
Length	UDP checksum
Data	

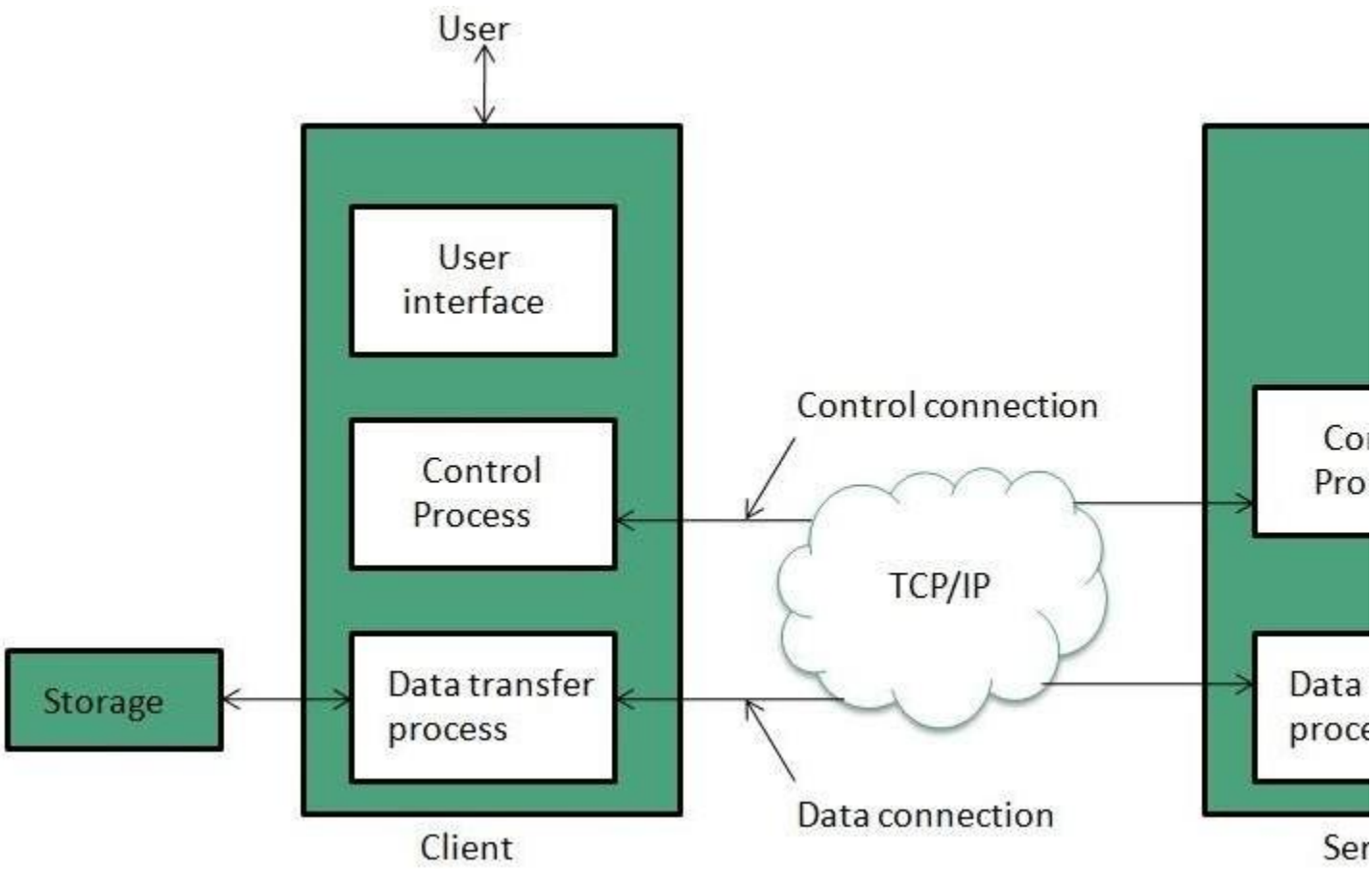
Points to remember:

- UDP is used by the application that typically transmit small amount of data at one time.
- UDP provides protocol port used i.e. UDP message contains both source and destination port number, that makes it possible for UDP software at the destination to deliver the message to correct application program.

File Transfer Protocol (FTP)

FTP is used to copy files from one host to another. FTP offers the mechanism for the same in following manner:

- FTP creates two processes such as Control Process and Data Transfer Process at both ends i.e. at client as well as at server.
- FTP establishes two different connections: one is for data transfer and other is for control information.
- **Control connection** is made between **control processes** while **Data Connection** is made between<="" b="">
- FTP uses **port 21** for the control connection and **Port 20** for the data connection.



Trivial File Transfer Protocol (TFTP)

Trivial File Transfer Protocol is also used to transfer the files but it transfers the files without authentication. Unlike FTP, TFTP does not separate control and data information. Since there is no authentication exists, TFTP lacks in security features therefore it is not recommended to use TFTP.

Key points

- TFTP makes use of UDP for data transport. Each TFTP message is carried in separate UDP datagram.
- The first two bytes of a TFTP message specify the type of message.
- The TFTP session is initiated when a TFTP client sends a request to upload or download a file.
- The request is sent from an ephemeral UDP port to the **UDP port 69** of an TFTP server.

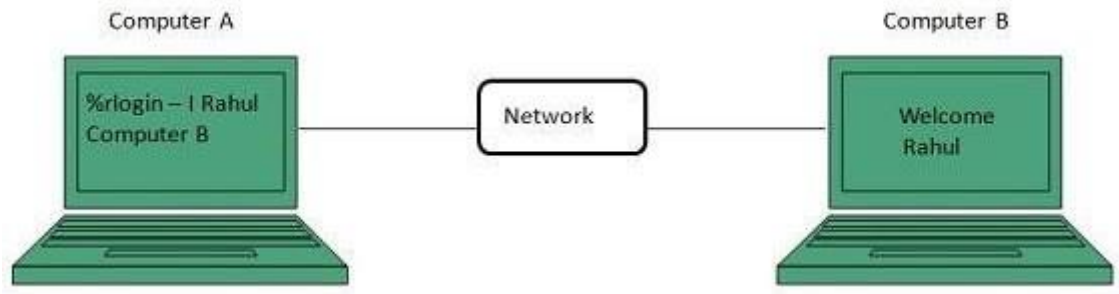
Difference between FTP and TFTP

S.N.	Parameter	FTP	TFTP
1	Operation	Transferring Files	Transferring Files

2	Authentication	Yes	No
3	Protocol	TCP	UDP
4	Ports	21 – Control, 20 – Data	Port 3214, 69, 4012
5	Control and Data	Separated	Separated
6	Data Transfer	Reliable	Unreliable

Telnet

Telnet is a protocol used to log in to remote computer on the internet. There are a number of Telnet clients having user friendly user interface. The following diagram shows a person is logged in to computer A, and from there, he remote logged into computer B.



Hyper Text Transfer Protocol (HTTP)

HTTP is a communication protocol. It defines mechanism for communication between browser and the web server. It is also called request and response protocol because the communication between browser and server takes place in request and response pairs.

HTTP Request

HTTP request comprises of lines which contains:

- Request line
- Header Fields
- Message body

Key Points

- The first line i.e. the **Request line** specifies the request method i.e. **Get** or **Post.**
- The second line specifies the header which indicates the domain name of the server from where index.htm is retrieved.

HTTP Response

Like HTTP request, HTTP response also has certain structure. HTTP response contains:

- Status line
- Headers
- Message body

What does Client/Server Architecture mean?

Client/server architecture is a computing model in which the server hosts, delivers and manages most of the resources and services to be consumed by the client. This type of architecture has one or more client computers connected to a central server over a network or internet connection. This system shares computing resources.

Client/server architecture is also known as a networking computing model or client/server network because all the requests and services are delivered over a network.

Client/server architecture is a producer/consumer computing architecture where the server acts as the producer and the client as a consumer. The server houses and provides high-end, computing-intensive services to the client on demand. These services can include application access, storage, file sharing, printer access and/or direct access to the server's raw computing power.

Client/server architecture works when the client computer sends a resource or process request to the server over the network connection, which is then processed and delivered to the client. A server computer can manage several clients simultaneously, whereas one client can be connected to several servers at a time, each providing a different set of services. In its simplest form, the internet is also based on client/server architecture where web servers serve many simultaneous users with website data.

.1 Client–server architecture

A **client–server architecture** (Figure 1) divides an application into two parts, 'client' and 'server'. Such an application is implemented on a computer network, which connects the client to the server. The server part of that architecture provides the central functionality: i.e., any number of clients can connect to the server and request that it performs a task. The server accepts these requests, performs the required task and returns any results to the client, as appropriate.

Consider an online bookstore as an example. The application allows a user to search and look at the details of a large range of books, and then to order a book. The application software provides an interface and a means of selecting or finding a book's details, as well as displaying book information and allowing a book order to be generated.

The application could take the form of a single 'chunk' of software downloaded from the web. However, if the software is one monolithic item, then every time anything is changed or updated, the entire application has to be redistributed again. Obviously this would not work well in this example because the catalogue of books will change regularly. An improvement might be to split the application into two parts. One part, the client, can provide the interface for users and be distributed to them. The other part can be kept and run on the company's own server machine (Figure 1). The client application can display information and be used to pass information to the server for searching, such as the title of a book. This client application, or software, is quite commonly called the 'presentation' layer or tier.

View larger image

Figure 1 Simple client–server application

Long description

In this client–server model, many clients can connect to the server application and request information about books. The server has to process these requests and send the response to the client that originated the request and not to any other client. As long as the network is working well and the server can keep up with responding to all the requests it receives, such a 'split' application will provide much the same level of service as the monolithic version. This simple client–server architecture is also commonly called 'two-tier architecture'.

The catalogue of book information can be held centrally on the server and then be easily updated. This allows other 'centralised' information to be maintained and sent to clients, such as the stock level of each book. Users of the client will find it much simpler and smaller to work with than the complete application. At the same time the company will have better control and be able to, for example, monitor usage of the server application itself. A common client used to access applications is a web browser that accesses server applications (such as applications on websites) using HTTP. The use of a web browser as the client end of an application is interesting because, for most applications, the browser is provided by a third party. This means that application builders must rely on agreed standards for the behaviour of the client component.

There are also different distributions of functionality across a two-tier architecture. For instance, suppose

you have a client that accesses your bank account online. If that client is a web browser, for example, it can be used to request and display your accounts' statements from the bank's server. The information you may obtain is restricted to the pre-defined views of the information provided by the server. So, while the server's information might include your account balance, if you want to find out the total payments in and out over the last week or year, then you will still have to calculate it yourself, based on the figures the server provides. Alternatively, you might access your bank server over the internet using another, 'more intelligent' client, such as a mobile app. This client might include a facility to extract figures from your bank statements and to perform whatever calculations you require. Such a client might also create bar or pie charts that display your income and expenditure across different categories that you define.

The web browser client in the example of the online bank simply displays the information that the server provides. The 'more intelligent' mobile app client allows you to take the information the server provides and to manipulate and display it in various ways according to your own personal needs. The web browser with little functionality of its own is often termed a **thin client** while the more intelligent client is usually termed a **thick (or 'thicker') client**.

We have seen that a two-tier approach seems to have some advantages, at least for applications that operate over networks. The client that is distributed to users may change, while the server part can be a centralised component that maintains dynamic, global data in a consistent and secure way for the organisation and for users to access and use. If a third party component, such as a web browser, provides the functionality required to support the application, then it can be adopted as part of the solution, with a significant saving of development effort. There is a potential disadvantage from splitting the application across a network in that data has to be transmitted over what may be a slow or unreliable connection.

There are other, slightly less obvious advantages in breaking the application into components in this way. We shall look at some of these in more detail shortly, but let's look at one advantage now. This is an advantage from which you will have benefited if you have ever decided to change your web browser. You might have changed from Internet Explorer to Mozilla Firefox, for example, just as a personal preference. When you did this you changed the client component of all the online applications you use. This is only possible because web browsers are based largely on common standards and because they are not realised as an intrinsic or built-in part of any of the applications you use; in other words, the client is loosely coupled to the server application.